Data 412/612: Midterm 1

Instructions

- Please place your answers in an R Markdown document.
- Submit this R Markdown document, and the resulting knitted PDF, to Canvas before the exam time has expired.
- Make sure all code is visible in the PDF by making sure `knitr::opts_chunk$set(echo = TRUE)` is in your first code chunk.
- You have three-hours and thirty minutes.
- You may use your notes and the course website. No Google, other websites, or LLMs.
- You may not discuss this exam with anyone (including over instant message or text).
- Use the tidyverse, not base R, where possible. Too much base R will suggests less understanding of the material in this class (since we haven't covered Base R).
- This exam has 100 points in total.

Question 1: Star Wars Characters

The `starwars` data frame in the dplyr package contains demographic characteristics of various characters from the hit franchise Star Wars. We will use this data frame for question 1.

1. (5 pts) Load the data into R and load all necessary packages for this exam (if you cannot figure out how to do this, I will tell you since it is required for the rest of this question, but you will not receive credit for this part).

```
library(tidyverse)
data("starwars")
```

2. (10 pts) Use appropriate R code to determine which individuals have missing gender. Make sure to only print out their names and heights, arranged in ascending order of height.

```
starwars %>%
  filter(is.na(gender)) %>%
  arrange(height) %>%
  select(name, height)
```

```
## # A tibble: 4 x 2
##   name             height
##   <chr>             <int>
## 1 Cordé               157
## 2 Sly Moore           178
## 3 Jek Tono Porkins    180
## 4 Gregar Typho        185
```

3. (10 pts) It seems all individuals in part 2 are droids. Change their gender to `"droid"` and save to the dataframe.

```
starwars %>%
  mutate(gender = replace(gender, is.na(gender), "droid")) ->
  starwars
```

4. (10 pts) The body mass index (BMI) is defined as

$$\frac{\text{mass}_{\text{kg}}}{\text{height}_m^2}.$$

That is, the BMI is the weight of the individual (in kg) divided by the height of the individual (in m) squared. There are 100 cm in each m.

Calculate the BMI for each individual in the `starwars` data frame and add it to the `starwars` data frame.

Sanity check: The median BMI should be in the 20-30 range.

```
starwars %>%
  mutate(bmi = mass / (height / 100) ^ 2) ->
  starwars
```

5. (10 pts) Calculate the median and mean height for each gender as well as the number of individuals.

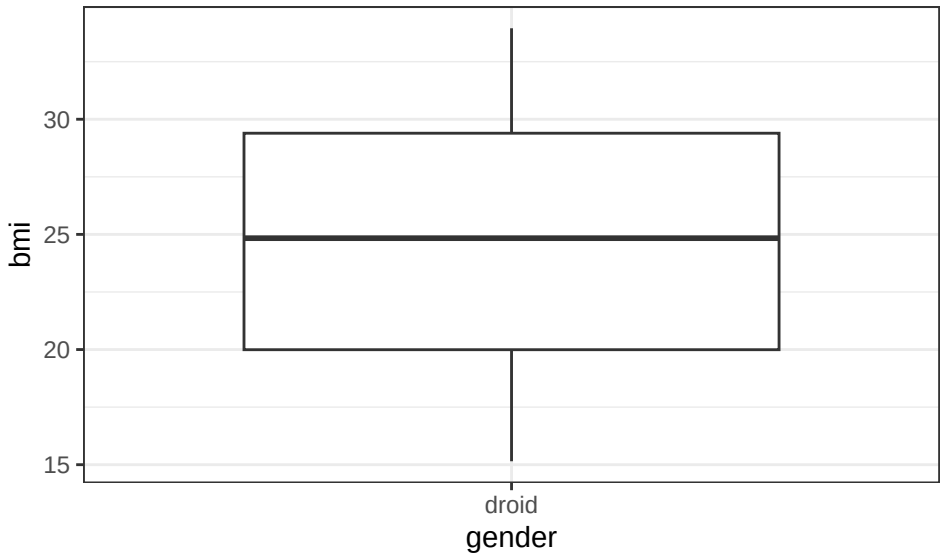
```
starwars %>%
  group_by(gender) %>%
  summarise(mean = mean(height, na.rm = TRUE),
            median = median(height, na.rm = TRUE), number = n())
```

```
## # A tibble: 3 x 4
##   gender     mean median number
##   <chr>     <dbl>  <dbl>  <int>
## 1 droid      175     179      4
## 2 feminine   167.    167     17
## 3 masculine  177.    183     66
```

6. (10 pts) Make boxplots of gender vs BMI. Include only males, females, and droids. Use the black and white theme.

```
starwars %>%
  filter(gender == "male" | gender == "female" | gender == "droid") %>%
  ggplot(mapping = aes(x = gender, y = bmi)) +
  geom_boxplot() +
  theme_bw()
```

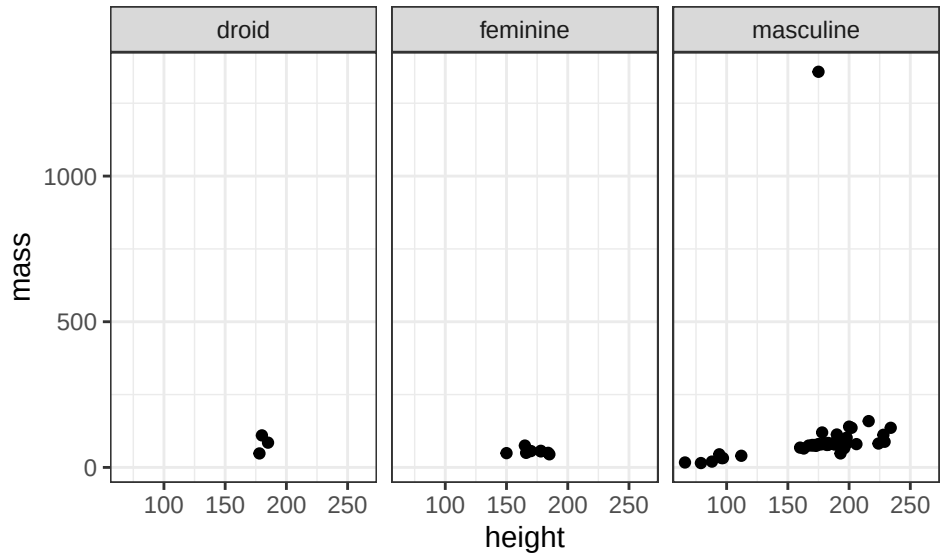
```
## Warning: Removed 1 rows containing non-finite values (‘stat_boxplot()‘).
```



7. (10 pts) Make scatterplots of height vs mass, faceting by gender.

```
ggplot(data = starwars, mapping = aes(x = height, y = mass)) +
  geom_point() +
  facet_wrap(. ~ gender) +
  theme_bw()
```

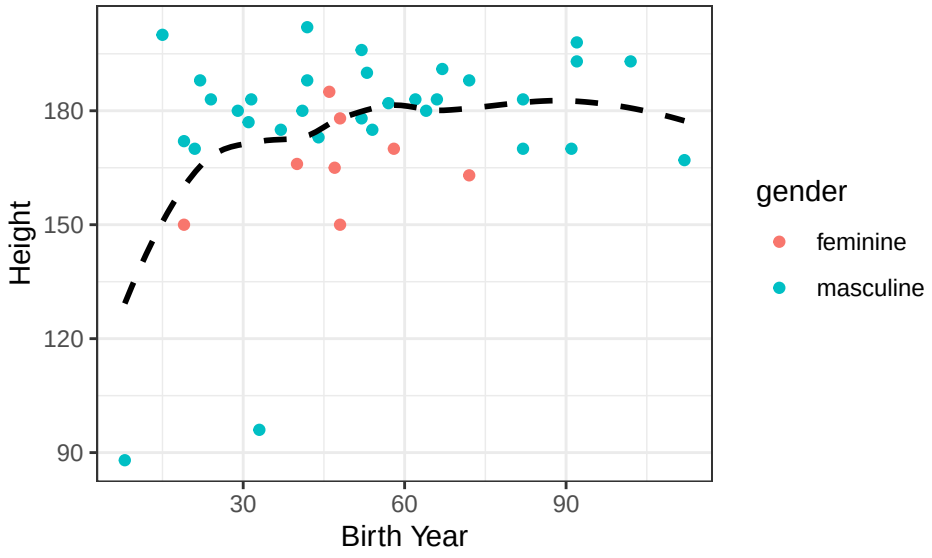
```
## Warning: Removed 28 rows containing missing values (`geom_point()`).
```



8. (10 pts) Reproduce the following plot (hint: filter based on birth year):

```
starwars %>%
  filter(birth_year < 200) %>%
  ggplot(mapping = aes(x = birth_year, y = height )) +
  geom_point(aes(color = gender)) +
```

```
  ylab("Height") +
  xlab("Birth Year") +
  geom_smooth(se = FALSE, color = "black", linetype = "dashed") +
  theme_bw()
```



Question 2: Coding Exercises

1. (10 pts) Write a function named `convert()` that takes one number as input and

- returns the square of an even whole number input,
- the cube of an odd whole number input, and
- `Inf` for a non-integer value.

For example, `convert(5)` should give `125`, `convert(8)` should give `64`, and `convert(4.7)` should give `Inf`. Evaluate your function at `0, 4, 7`, and `2.5`.

```r
convert <- function(x) {
  if(x %% 2 == 0) {
    return(x ^2)
  } else if (x %% 2 == 1) {
    return(x ^ 3)
  } else {
    return(Inf)
  }
}
convert(0)
```

```
## [1] 0
```

```r
convert(4)
```

```
## [1] 16
```

```r
convert(7)
```

```
## [1] 343
```

```r
convert(2.5)
```

```
## [1] Inf
```

2. (7.5 pts) Reproduce the following line (with URL) and table in R Markdown:

   The **quick** brown ~~fox~~ `jumps`

over	the
lazy	dog

3. (7.5 pts) **Challenge** (do this one last): Write a function called `evenodd()` that takes a single positive integer as input.

   It then prints a data frame where one variable is a sequence from 1 to the given number and the other variable describes whether the number is even or odd. If the number is less than or equal to 0, it should return an error with a specific error message. Evaluate your function at `5`, `8`, and `-100` (in the code chunk options, set `error = TRUE` to allow for errors while knitting).

   For example, the following are some evaluations of my implementation of `evenodd()`. Evalute at 5, 6, -1.

```r
evenodd <- function(n) {
      if (n <= 0) {
        stop("n is less than 1")
      }
      dfdat <- tibble(number = seq_len(n),
                      type = rep(c("odd", "even"), length.out = n))
      return(dfdat)
    }
```

```r
evenodd(5)
```

```
## # A tibble: 5 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
## 4      4 even
## 5      5 odd
```

```r
evenodd(6)
```

```
## # A tibble: 6 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
## 4      4 even
## 5      5 odd
## 6      6 even
```

```r
evenodd(-1)
```

```
## Error in evenodd(-1): n is less than 1
```

```r
evenodd(3)
```

```
## # A tibble: 3 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
```

```r
evenodd(4)
```

```
## # A tibble: 4 x 2
##   number type
##    <int> <chr>
## 1      1 odd
## 2      2 even
## 3      3 odd
## 4      4 even
```

```r
evenodd(0)
```

```
## Error in evenodd(0): n is less than 1
```